

Nível 1

Objetivo

Adicionar um endpoint de consulta simples e um script de teste.

Tarefas

1. Criar endpoint na API:
 - o `GET /purchases/recent?limit=20`
 - o Retorna as **últimas N compras** ordenadas por `ts desc`.
2. Criar script:
 - o `scripts/recent.sh` chamando o endpoint com `curl`.

Regras

- Não pode usar SQL “na mão” no controller.
- Implementar via `PurchaseRepository` (Spring Data JPA).
- Deve compilar e rodar via `docker compose up`.

Critério de aceite

- `curl http://localhost:8080/purchases/recent?limit=5` retorna JSON com 5 registros.
- Script `scripts/recent.sh` executa e imprime resultado.

Nível 2

Objetivo

Criar uma query agregada com filtro e adicionar cache com invalidação/TTL coerente.

Tarefas

1. Novo endpoint:
 - o `GET /analytics/customer-totals?sinceMinutes=60`
 - o Retorna total por cliente **apenas das compras** nos últimos X minutos.
2. Ajustar banco:
 - o Criar um índice útil para o filtro por tempo (você já tem `idx_purchases_ts`, mas peça para eles justificarem se mantém ou muda para composto).
3. Cache:
 - o Cachear esse endpoint por **10 segundos** (novo cache separado).
 - o Nome do cache: `customerTotalsSince`.

Regras

- Cache TTL deve ser configurado em `CacheConfig`.
- Endpoint deve ser estável e idempotente.
- Explicar (em 3 bullets) **por que cache faz sentido nesse caso**.

Critério de aceite

- Requisições repetidas em $< 10s$ retornam mais rápido e não “rebatem” no DB (pelo menos perceptível no live).
- Após 10s, resultado pode mudar conforme o worker grava.

Nível 3

Objetivo

Evoluir o worker para um cenário mais “produção”: idempotência, erros e rastreabilidade.

Tarefas

1. **Idempotência do Worker**
 - Simular que o worker pode “reprocessar” um evento.
 - Regra: se tentar inserir `id` já existente, não pode quebrar o processo.
 - Saída: log indicando “duplicate ignored”.
2. **Dead Letter (DLQ) simplificado**
 - Criar tabela `failed_events` via Flyway (nova migration `v2__failed_events.sql`).
 - Quando der erro ao salvar compra (simule erro: amount negativo 1% das vezes), gravar o payload em `failed_events`.
3. **Observabilidade mínima**
 - Adicionar endpoint na API:
 - `GET /ops/failed-events/count`
 - Retorna contagem de eventos em DLQ.

Regras

- Não pode “engolir” exception sem registrar (log + persistir em `failed_events`).
- Migration deve ser via Flyway (no módulo `api`, que já roda as migrations).
- Worker precisa continuar rodando mesmo com falhas ocasionais.

Critério de aceite

- Worker não cai quando dá falha.
- `failed_events` cresce ao longo do tempo.
- Endpoint retorna o count corretamente.